

Lab 02 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: eerst het antwoord zelf, dan het mechanisme erachter, een nuance of valkuil, en een voorbeeld dat je aan de terminal kunt controleren.

Vraag 1 — Volledige namen en korte namen

Antwoord.

Schrijfwijze	Volledige naam	Podman
nginx	docker.io/library/nginx:latest	gekende alias → opgelost zonder vraag, met de melding Resolved "nginx" as an alias
bitnami/nginx	docker.io/bitnami/nginx:latest	geen alias → opgezocht in unqualified-search-registries; Ubuntu heeft er maar één (docker.io), dus opgelost; Fedora heeft er meerdere, dus stelt Podman een vraag
registry.mijnbedrijf.be:5000/basis/nginx:1.25	ongewijzigd	volledige naam: niets op te lossen

De regel is eenvoudig: bevat het eerste deel (vóór de eerste /) een **punt** of een **dubbelpunt** (poort), of is het localhost, dan is het een registry. Anders is het een namespace op de standaardregistry.

Waarom. mijnbedrijf.be kan geen gebruikersnaam op Docker Hub zijn (punten zijn daar niet toegelaten), en een poort heeft alleen zin voor een server. Docker past die regel toe en vult de naam dan stilzwijgend aan. Podman past dezelfde regel toe maar weigert blindelings aan te vullen, want nginx op docker.io en nginx op registry.intern kunnen twee verschillende images zijn.

Nuance. Ondanks haar naam is bitnami/nginx **geen** officiële image — de namespace is bitnami, niet library. En een image die je bouwt zonder een registry te noemen, wordt localhost/...: ook dat is een volledige naam, met localhost als fictieve "registry".

Voorbeeld.

```
podman pull nginx 2>&1 | head -2          # Resolved "nginx" as an alias ...
docker.io/library/nginx:latest
podman image inspect --format '{{.RepoTags}}' nginx
podman build -t api:1.0 . && podman images | grep api    # localhost/api 1.0
```

Vraag 2 — Zelfde tag, andere inhoud

Antwoord. Iemand heeft de tag `2.3` intussen **verplaatst**: een image opnieuw gebouwd en onder dezelfde naam gepusht. Server A hield de oude image (er is nooit opnieuw gepuld); server B kreeg de nieuwe. Je bewijst het door de digests te vergelijken. Vermijden doe je door een gepubliceerde tag nooit te hergebruiken en door op digest uit te rollen.

Waarom. Een tag is aan registryzijde een muteerbare pointer. Een `pull` vergelijkt de digest op de registry met de lokale digest en downloadt alleen wanneer ze verschillen. Niets waarschuwt je dat een tag verplaatst is.

Nuance. De verplaatsing kan onbedoeld zijn: een pipeline die bij elke run op de releasebranch `api:2.3` pusht, of een `latest`. Ook de basisimage kan verplaatst zijn zonder dat je Dockerfile verandert (`FROM eclipse-temurin:21-jre`): "dezelfde code" opnieuw bouwen levert dan een andere image op.

Voorbeeld.

```
# op A en op B:
podman image inspect --format '{{.Digest}}' mijnapp/api:2.3
# verschillende digests -> de tag is bewogen. Correcte uitrol:
podman pull registry.intern/mijnapp/api@sha256:9d0d1f1e...
```

Vraag 3 — 62 GB getoond, schijf intact

Antwoord. Nee. `SIZE` toont de **virtuele** grootte van elke image, gedeelde lagen inbegrepen: gemeenschappelijke lagen (JRE, Alpine, Debian) worden bij elke image meegeteld maar staan maar één keer op schijf. `podman system df` toont het echte verbruik. De bestanden staan in `~/.local/share/containers/storage` — binnen de virtuele schijf van de WSL-distributie (`ext4.vhdx`), niet in een Windows-map.

Waarom. Het stuurprogramma `overlay` slaat elke laag één keer op, geïdentificeerd op inhoud, en images zijn niets meer dan lijsten van lagen. Twaalf Spring Boot-images op dezelfde JRE delen die 180 MB.

Nuance. De `vhdx` van WSL **groeit** automatisch maar **krimpt** nooit vanzelf wanneer je images verwijdt. De ruimte komt vrij aan Linux-zijde, niet aan Windows-zijde, tot je de schijf compacteert (`wsl --shutdown` en daarna `Optimize-VHD` of `diskpart`). Daar kijken veel Windows-gebruikers van op.

Voorbeeld.

```
podman system df                # echte SIZE en RECLAIMABLE
podman system df -v | head      # kolom SHARED SIZE per image
podman info --format '{{.Store.GraphRoot}}' # /home/<jij>/.local/share/containers/storage
```

Vraag 4 — De `rm` die niets verwijdt

Antwoord. Hij heeft ongelijk. `COPY` maakt een laag aan die `credentials.json` **bevat**. De `RUN ... rm` maakt een latere laag aan met een *whiteout* die het bestand verbergt. De uiteindelijke image bevat beide lagen: het bestand zit er nog in, het is alleen onzichtbaar vanuit een container.

Waarom. Lagen zijn onveranderlijk en additief. Een laag kan geen bestand uit een eerdere laag wegnemen; ze kan het alleen verbergen. Wie de image heeft, kan er `podman save` op loslaten, de

laag van de `COPY` uitpakken en het bestand lezen.

Nuance. De stappen samenvoegen verandert hier niets: ook al volgt de `rm` meteen op de kopie, de `COPY` blijft een aparte instructie met een eigen laag. Alleen een *build secret* (`RUN --mount=type=secret`) of een multi-stage build (lab 05) houdt het bestand uit de uiteindelijke image.

Voorbeeld.

```
podman save --format oci-archive -o img.tar mijn-image:1.0
mkdir x && tar -xf img.tar -C x
for b in x/blobs/sha256/*; do tar -tf "$b" 2>/dev/null | grep -q credentials.json && echo
"aanwezig in $b"; done
```

Vraag 5 — 310 MB, 61 MB overgedragen

Antwoord. De registry heeft de ongewijzigde lagen al (JRE, systeem, afhankelijkheden), dus de `push` verstuurt alleen de nieuwe — de JAR en alles daarna, samen 61 MB. Zou elke push de volle 310 MB versturen, dan betekent dat dat de eerste lagen van de image bij elke build veranderen: een `COPY . .` die te vroeg staat, of een instructie met variabele inhoud (een datum, een versienummer) vóór de zware lagen.

Waarom. Elke laag heeft een digest. Voor het uploaden van een blob vraagt de client aan de registry of ze hem al heeft (`HEAD /v2/<repo>/blobs/<digest>`). Podman toont dat minder uitdrukkelijk dan Docker (geen regel `Layer already exists`), maar de push is bijna meteen klaar.

Nuance. Of lagen gedeeld worden tussen *repositories* van dezelfde registry, hangt af van de implementatie (Harbor en Docker Registry doen het via *cross-repository mount*). En een "ongewijzigde" laag moet bit voor bit identiek zijn: een `RUN apt-get update` zonder vastgepinde versies levert bij elke build een andere laag op.

Voorbeeld.

```
podman push --tls-verify=false localhost:5000/basis/demo:1.1 # onmiddellijk: blobs al
aanwezig
podman history mijn-api:2.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # de laag die verandert
opsporen
```

Vraag 6 — Twee tags, één image, en een spook

Antwoord. Er zijn twee verschillende images: `f3a1b9c02d11` (met de twee tags `2.0` en `1.9`) en `8b2c74e91a03` (zonder naam). De regel `<none>` is de *dangling* image: een tag (waarschijnlijk `2.0`) is opnieuw gebouwd en doorgeschoven, waardoor de oude image haar naam verloor. `podman rmi api:1.9` verwijdert **alleen de tag** (`Untagged: localhost/api:1.9`); de data blijft staan en blijft bereikbaar via `2.0`. En `localhost/` is de fictieve registry van elke image die zonder registrynaam gebouwd of getagd is.

Waarom. Een `IMAGE ID` is de digest van de imageconfiguratie; twee regels met dezelfde ID zijn twee namen voor één inhoud. De data verdwijnt pas samen met de laatste naam.

Nuance. Verwar *dangling* (`<none>:<none>`, helemaal geen tag) niet met *unused* (wel een tag, maar geen container). Een `podman image prune` zonder `-a` verwijdert alleen de eerste soort.

Voorbeeld.

```
podman rmi api:1.9 # Untagged: localhost/api:1.9 (geen Deleted)
podman images --filter dangling=true -q # 8b2c74e91a03
podman rmi 8b2c74e91a03 # Deleted: ... deze keer verdwijnt de data
```

Vraag 7 — exec format error

Antwoord. De image is gebouwd voor `linux/arm64` (Apple Silicon) en de server draait `linux/amd64`: de kernel kan het binary niet uitvoeren. Om meteen te deblokken: opnieuw bouwen met `--platform linux/amd64` (QEMU-emulatie — traag, maar het werkt). Om het definitief op te lossen: de images door de **CI** laten bouwen op `amd64`-agents, of multi-architectuur-images publiceren (`podman manifest`).

Waarom. Een multi-arch-tag is een manifest list. Bij de `build` produceert de engine een manifest voor de architectuur van de bouwmachine. Bij de `pull` wordt de `amd64`-ingang gezocht — die bestaat niet, dus komt de `arm64`-image binnen.

Nuance. Bijna alle *officiële* images zijn multi-arch, waardoor het probleem verborgen blijft tot de eerste eigen build. De fout kan zich ook eerder en stiller tonen: een `amd64`-image draaien op de Mac lukt wél (via emulatie), alleen 5 tot 10 keer trager.

Voorbeeld.

```
podman image inspect --format '{{.Architecture}}' registry.intern/api:1.4 # arm64
podman build --platform linux/amd64 -t registry.intern/api:1.4 .
```

Vraag 8 — save tegenover export, en het OCI-formaat

Antwoord. `save` exporteert een **image**: lagen, manifest, configuratie, tags. `export` exporteert het bestandssysteem van een **container**, platgeslagen tot één boom, zonder metadata. Om een Spring Boot-image naar een afgesloten site te brengen is `save` dus de juiste keuze. Met `export` verlies je `ENTRYPOINT`, `CMD`, `ENV`, `EXPOSE`, `USER` en `WORKDIR` — de geïmporteerde image weet niet meer hoe ze moet starten — en daarbovenop de lagen zelf, dus geen delen en geen cache meer. `--format oci-archive` levert dezelfde image in de OCI-standaardindeling (`blobs/sha256/`, `index.json`) in plaats van het historische Docker-formaat.

Waarom. `export` ziet alleen het eindresultaat van de samengevoegde lagen, zoals een `tar` die je binnenin de container zou nemen. De configuratie leeft in de image, niet in het bestandssysteem.

Nuance. Toch heeft `export` zijn nut: een bestandssysteem ophalen voor analyse, of een "platte" image maken van een handmatig geconfigureerde container (slechte praktijk, maar gedocumenteerd). Het formaat `docker-archive` blijft het meest gangbare; `oci-archive` gebruik je wanneer de ontvanger geen Docker is (Kubernetes via `ctr`, skopeo...).

Voorbeeld.

```
podman save --format oci-archive -o api.tar mijn-api:1.0
podman load -i api.tar # Loaded image: localhost/mijn-api:1.0
podman export c1 | podman import - plat:1 # Config.Cmd = null
```

Vraag 9 — De tweede `pull`

Antwoord. De engine heeft de registry om het **manifest** van de tag gevraagd, de digest ervan vergeleken met die van de lokale image, vastgesteld dat ze identiek zijn, en verder niets gedownload. Een manifest is enkele kilobytes groot; de netwerkkost blijft beperkt tot een of twee HTTP-verzoeken.

Waarom. Het manifest en elke laag worden op inhoud geadresseerd, dus de client weet precies wat hij al heeft. Een pull is altijd differentieel — in het uiterste geval wordt er niets overgedragen.

Nuance. Podman zegt nooit "Image is up to date"; het print gewoon de image-ID. "Minder dan een seconde" veronderstelt bovendien een registry dichtbij: tegenover Docker Hub kan het verzoek enkele seconden latentie kosten, zonder dat er iets gedownload wordt. En dat verzoek telt wél mee voor het quotum van Docker Hub (vraag 10).

Voorbeeld.

```
time podman pull alpine      # d529dd0c... — enkele seconden netwerk, nul lagen overgedragen
```

Vraag 10 — `toomanyrequests`

Antwoord. Docker Hub begrenst anonieme `pull`s per IP-adres (en per account voor aangemelde gebruikers). Alle CI-agents verlaten het netwerk via hetzelfde publieke IP, dus het hele bedrijf deelt één quotum — daarom lijken de fouten "willekeurig": het hangt ervan af hoeveel er op dat moment al gepulld is. De twee klassieke oplossingen: een **pull-through cache** (een interne registry die Docker Hub cachet) en/of een **interne kopie** van de basisimages in de bedrijfsregistry (`skopeo copy`), met Dockerfiles die naar die registry verwijzen.

Waarom. Elke `pull` bevraagt minstens het manifest, zelfs wanneer de image al lokaal staat. Een CI die honderd keer per dag bouwt, jaagt er het quotum snel door.

Nuance. Aanmelden (`podman login docker.io`) verhoogt het quotum maar heft het niet op — en het zet een persoonlijk account in de CI. De interne kopie heeft een extra voordeel: je controleert *wat binnenkomt* (scan, validatie) en je hangt niet langer af van de beschikbaarheid van Docker Hub.

Voorbeeld.

```
skopeo copy docker://docker.io/library/node:22-alpine docker://registry.intern/basis/node:22-alpine
# en dan in de Dockerfile: FROM registry.intern/basis/node:22-alpine
```

Vraag 11 — HTTP tegenover HTTPS

Antwoord. Docker behandelt `localhost` (en `127.0.0.0/8`) standaard als een *insecure* registry: het aanvaardt gewone HTTP zonder commentaar. Podman kent die uitzondering niet: elke registry moet een geldig TLS-certificaat voorleggen. Er zijn twee manieren om erdoor te raken: `--tls-verify=false` meegeven op het commando, of een ingang `[[registry]] location = "localhost:5000" insecure = true` toevoegen in `registries.conf`. De tweede mag nooit in een geversioneerd bestand of op een server belanden: ze schakelt de verificatie stilzwijgend uit voor **elk** gebruik van die registry.

Waarom. Wie op het netwerkpad zit, kan een registry zonder TLS nabootsen (*man in the middle*) en een besmette image terugsturen. Op `localhost` is dat risico klein — maar Podman wil dat je

dat expliciet zegt in plaats van het stilzwijgend aan te nemen.

Nuance. `--tls-verify=false` schakelt ook de **certificaat**verificatie uit bij een zelfondertekende HTTPS-registry. Beter is het certificaat van de interne autoriteit te installeren in `/etc/containers/certs.d/<registry>/ca.crt`.

Voorbeeld.

```
podman push --tls-verify=false localhost:5000/basis/demo:1.0      # expliciet, zichtbaar in de
geschiedenis
# OF, alleen voor een ontwikkelmachine:
printf '[[registry]]\nlocation = "localhost:5000"\ninsecure = true\n' >>
~/.config/containers/registries.conf
```

Vraag 12 — image is in use by a container

Antwoord. Er bestaat nog een container die van die image gemaakt is — mogelijk gestopt — en de image is zijn basislaag. De engine weigert omdat de image verwijderen die container zou breken. De nette aanpak: lijst de container op (`podman ps -a --filter ancestor=...`), verwijder hem zodra je zijn toestand niet meer nodig hebt, en doe dan pas `rmi`. `podman rmi -f` verwijdert de tag, de image **én** de afhankelijke container zonder iets te vragen: je verliest de logs, de schrijflaag en elke kans om de container nog te inspecteren — en dat om tien seconden te winnen.

Waarom. Een container is een image plus een schrijflaag. Zonder de image betekent de schrijflaag niets meer.

Nuance. De melding van Podman heeft het over "externe containers": containers die door Buildah of een andere tool met dezelfde opslag zijn aangemaakt en die `podman ps` niet toont. `podman ps -a --external` toont ze wel. Docker weigert in dezelfde situatie, maar zijn melding vermeldt de container-ID voluit.

Voorbeeld.

```
podman ps -a --filter ancestor=mijn-api:1.0 --format '{{.Names}} {{.Status}}'
podman logs <container> > incident.log      # we bewaren wat bewaard moet worden
podman rm <container> && podman rmi mijn-api:1.0
```

Vraag 13 — De lagen van 0B

Antwoord. Ze komen van **metadata**-instructies — `ENV`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `LABEL`, `USER`, `WORKDIR` — die de configuratie van de image wijzigen zonder het bestandssysteem aan te raken. Ze verschijnen in de geschiedenis omdat elke instructie een spoor nalaat, ook een leeg spoor. De laag van 180 MB (een `RUN apt-get install`, een `COPY` van een JRE) is de enige die telt: `podman history` wijst meteen de regel aan die je moet optimaliseren.

Waarom. Een image is een lijst instructies, waarvan sommige een blob met bestanden meedragen. Al het gewicht zit in die blobs.

Nuance. Een niet-lege laag kan een verwijdering verbergen: een `RUN rm -rf /var/lib/apt/lists/*` in een aparte `RUN` weegt bijna 0B maar levert niets op. `history` toont wat elke stap kost; `podman image tree` toont daarbovenop welke lagen van de basisimage komen.

Voorbeeld.

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}' # 12 regels van 0B,
één van 50.7MB
podman image tree nginx:alpine
```

Vraag 14 — Drie tagstrategieën

Antwoord. (a) Een overschreven `latest` maakt rollback onmogelijk — de oude image heeft geen naam meer — en diagnose evenmin, want niemand kan nog achterhalen welke versie draaide. (b) Met `1.4.2` rol je onmiddellijk terug naar `1.4.1`, en diagnose lukt zolang de tag onveranderlijk blijft; maar twee builds van `1.4.2` (na een snelle fix, bijvoorbeeld) kunnen naast elkaar bestaan zonder dat je ze uit elkaar kunt houden. (c) `1.4.2-b318-a9f3c21` maakt elke image uniek: je kunt terug tot de exacte commit en build, en terugrollen naar eender welke eerdere build. Daar staat een onleesbare naam tegenover, plus een retentiebeleid om te beheren.

Waarom. Uitrol en diagnose hebben allebei een **één-op-één**-verband nodig tussen een naam en een inhoud. Alleen (c) garandeert dat vanzelf; bij (b) hangt het af van discipline; met (a) is het uitgesloten.

Nuance. De drie bestaan vaak naast elkaar: de CI pusht (c); een tag (b) wordt aan dezelfde image *toegevoegd* zodra ze gevalideerd is; `latest` bestaat alleen voor het gemak van de ontwikkelaars en komt nooit in een uitrolmanifest. En de uitrol zelf pint de digest vast.

Voorbeeld.

```
podman tag registry.intern/api:1.4.2-b318-a9f3c21 registry.intern/api:1.4.2 # zelfde image,
tweede naam
podman image inspect --format '{{.Digest}}' registry.intern/api:1.4.2 # wat werkelijk
uitgerold wordt
```